

Java Variables & Data Types

Java Variables & Data Types

1. Introduction
2. What Is a Variable?
3. Identifiers, Literals & Naming
4. Data Types & Static Typing
5. Primitive Data Types — Overview
6. How Numbers Are Stored: Binary Basics
7. Integer Types — byte, short, int, long
8. Floating-Point Types — float, double
9. Character Type — char
10. Boolean Type
11. Number Systems in Java Literals
12. Underscores in Large Numeric Literals
13. Declaration vs. Definition
14. Keywords
15. Comments

★ One-Page Revision

📝 Practice Questions

🔥 Frequently Asked Questions

Java Variables & Data Types

Identifiers, Literals, Keywords — Lecture 3

1. Introduction

This lecture moves from Java basics into actual syntax — specifically **how Java stores data**.

What you'll learn:

- What a variable really is (and why we need it)
- Java's primitive data types and their memory ranges
- How numbers are stored in binary
- Identifiers, literals, and keywords
- Number systems: decimal, binary, octal, hexadecimal
- Declaration vs. definition
- Comments

Why it matters: Every single Java program you'll ever write stores and manipulates data — you can't write a calculator, a form, or an app without variables. Getting this foundation solid makes everything after it (loops, methods, objects) much easier.

2. What Is a Variable?

Definition: A variable is a **named container** in computer memory that holds a value.

Why we need variables: Imagine building a simple calculator that adds two numbers. Before you can add them, you need to **store** them somewhere in memory. A variable is simply a labeled spot in memory where a value lives.

Computer Memory

```
firstNumber = 4  
secondNumber = 3
```

Instead of tracking raw memory addresses, we give each memory location a **name** — this name is called an **identifier**.

💡 Easy Hinglish Explanation

"Variable ek container hota hai jisme hum koi value store karte hain — jaise ek dabba jisme tumne kuch rakha hai. Us dabbe ka naam hi 'identifier' hota hai, aur jo cheez usme rakhi hai wo 'literal' kehlaati hai."

3. Identifiers, Literals & Naming

Term	Meaning	Example
Identifier	The name given to a variable	<code>firstNumber</code>
Literal	The actual value stored	<code>10</code> , <code>'a'</code> , <code>true</code>

★ **Best Practice** Always use **descriptive names**. `x` and `y` tell you nothing — `firstNumber` and `secondNumber` tell you exactly what's stored, making your code self-explanatory.

4. Data Types & Static Typing

Before storing any value, Java needs to know **what kind of value** it will be — a whole number? A decimal? A single character? This is called the **data type**.

Definition: Java is a **statically typed language** — you must declare a variable's data type before (or at) compile time. You can't leave it undecided.

Java data types split into two broad categories:

```

Data Types
├── Primitive      (byte, short, int, long, float, double, char, boolean)
└── Non-Primitive (covered later, under Object-Oriented Programming)
  
```

📌 **Important:** This lecture only covers **primitive** data types. Non-primitive types (arrays, classes, objects) come later once OOP is introduced.

5. Primitive Data Types — Overview

Java groups its 8 primitive types into 4 categories:

Category	Types	Holds
Integer	<code>byte</code> , <code>short</code> , <code>int</code> , <code>long</code>	Whole numbers
Floating-point	<code>float</code> , <code>double</code>	Decimal (real) numbers
Character	<code>char</code>	A single character
Boolean	<code>boolean</code>	<code>true</code> or <code>false</code> only

6. How Numbers Are Stored: Binary Basics

Computers only understand **0s and 1s**. Every value — no matter the data type — eventually gets converted into binary.

Place values in binary (base 2)

Just like decimal has 1s, 10s, 100s place (powers of 10), binary has 1s, 2s, 4s, 8s place (powers of 2):

Binary Place Values:

128	64	32	16	8	4	2	1
(2 ⁷)	(2 ⁶)	(2 ⁵)	(2 ⁴)	(2 ³)	(2 ²)	(2 ¹)	(2 ⁰)

Example — representing 10:

0	0	0	0	1	0	1	0	→ 8 + 2 = 10
---	---	---	---	---	---	---	---	--------------

The range formula

★ **Exam Tip — Remember this formula** With **n bits**, you can form **2ⁿ unique combinations**. The **largest unsigned value** representable = **2ⁿ – 1** Since Java numbers are **signed** (can be negative or positive), the range splits roughly in half: from **-2ⁿ⁻¹** to **2ⁿ⁻¹ – 1**

💡 **Easy Hinglish Explanation**

"Agar tumhare paas n bits hain, to tum 2 ki power n tak unique numbers bana sakte ho. Lekin Java me har number signed hota hai — matlab positive aur negative dono represent karne hote hain — isliye range aadhi negative side aur aadhi positive side me baant di jaati hai."

7. Integer Types — byte, short, int, long

Type	Size (bits)	Range	Typical Use
<code>byte</code>	8	−128 to 127	Small numbers, memory-efficient
<code>short</code>	16	−32,768 to 32,767	When <code>byte</code> isn't enough
<code>int</code>	32	-2^{31} to $2^{31} - 1$	Default choice for most numbers
<code>long</code>	64	-2^{63} to $2^{63} - 1$	Very large numbers

Code Example

```
byte b = 5;
short s = 10;
int i = 4000;
long l = 10000;

System.out.println("Integer Values: " + b + ", " + s + ", " + i + ", " +
l);
```

Output:

```
Integer Values: 5, 10, 4000, 10000
```

Line-by-line:

- Each declaration follows: `dataType identifier = literal;`
- `b`, `s`, `i`, `l` are identifiers; `5`, `10`, `4000`, `10000` are literals
- A value that fits in `byte` (like `5`) could also be stored in `short`, `int`, or `long` — the smaller type is just more memory-efficient

⚠ Common Mistake

Don't assume you must always use the *smallest possible* type. `int` is the safe, default choice for everyday numbers unless you have a specific memory-optimization reason.

8. Floating-Point Types — float, double

Type	Size (bits)	Precision	Typical Use
float	32	Single precision	Legacy code, low-precision needs
double	64	Double precision	Default for decimal numbers

Code Example

```
float f = 10.54f;  
double d = 23.0987;  
  
System.out.println("Floating Values: " + f + ", " + d);
```

Output:

```
Floating Values: 10.54, 23.0987
```

 **Important** Any decimal literal (like `10.54`) is treated as `double` **by default**. If you want to store it in a `float` variable, you must add an `f` suffix — otherwise Java throws a type-mismatch error.

Scientific notation

Large or small decimal numbers can also be written in scientific form:

```
double d = 6.022e23;    // same as 6.022 × 1023 (Avogadro's number)
```

9. Character Type — char

Definition: `char` stores a **single character**, written inside single quotes: `'a'`.

```
char c = 'a';  
System.out.println("Character Value: " + c);
```

Output: `Character Value: a`

Why single quotes matter

Without quotes, Java can't tell if you meant the character `a` or an identifier named `a`. The quotes tell the compiler: *"this is a character literal."*

How a character is actually stored

A character is never stored as itself — it's converted to an **integer**, and that integer is stored in binary.

```
'a' → 65 (its numeric code) → 01000001 (binary)
```

Whether Java prints back `65` or `'a'` depends entirely on the variable's declared data type (`char` vs `int`).

Unicode vs. ASCII

Standard	Bits Used	Character Range
ASCII (old standard)	8-bit	English letters, digits, symbols only
Unicode (Java uses this)	16-bit	Every language in the world

💡 Easy Hinglish Explanation

"ASCII sirf English characters ko represent kar paata tha — 8 bits mein. Lekin Java duniya bhar mein use hone wali language banana chahti thi, isliye usne Unicode use kiya — 16 bits, jisme Hindi, Chinese, Tamil, koi bhi language ka character represent ho sakta hai."

10. Boolean Type

Definition: `boolean` can only hold **two values**: `true` or `false`. Nothing else.

```
boolean isValid = true;
```

⚠️ Common Mistake

Coming from a C/C++ background? In those languages, any non-zero number means `true` and `0` means `false`. **This does NOT work in Java.** You cannot write `boolean b = 1;` — Java strictly requires the literal `true` or `false`.

11. Number Systems in Java Literals

Beyond plain decimal, Java lets you write integer literals in **binary, octal, and hexadecimal** too.

Number System	Base	Prefix	Digits Allowed
Decimal	10	(none)	0–9
Binary	2	<code>0b</code>	0–1
Octal	8	<code>0</code>	0–7
Hexadecimal	16	<code>0x</code>	0–9, a–f

Code Examples (all represent the value 5)

```
byte b1 = 5;           // Decimal
byte b2 = 0b101;       // Binary   → 0b prefix
byte b3 = 05;          // Octal    → 0 prefix
byte b4 = 0x5;         // Hexadecimal → 0x prefix

System.out.println(b1 + " " + b2 + " " + b3 + " " + b4);
```

Output: `5 5 5 5` — all four print the same decimal value, because internally Java always converts them to the same binary representation.

Hexadecimal's special digits

Hex needs symbols for values 10–15, since single digits only go up to 9:

Decimal	10	11	12	13	14	15
Hex	a	b	c	d	e	f

⚠ Common Mistake

Each number system has a strict digit range. Writing `08` as an octal literal (`0` prefix) causes a compile error, because octal only allows digits 0–7. Similarly, `0b2` is invalid — binary only allows 0 and 1.

12. Underscores in Large Numeric Literals

For readability, Java lets you insert underscores (`_`) inside large numbers. The compiler ignores them completely.

```
long bigNumber = 1_23_45_67_890L;
System.out.println(bigNumber);
```

Output: `1234567890`

⚠ Common Mistake

You **cannot** place an underscore immediately before/after a decimal point, or immediately before/after an `e` in scientific notation. Everywhere else is fine.

13. Declaration vs. Definition

Term	Meaning	Example
Declaration	Telling the compiler a variable's type and name will exist	<code>int x;</code>
Definition	Assigning it an actual value	<code>x = 4;</code>

You can declare and define **separately**:

```
int x;           // Declaration – memory reserved, no value yet
x = 4;          // Definition – value assigned
```

...or **together** in one line, which is what we've done so far:

```
int x = 4;      // Declaration + Definition combined
```

🔴 **Important:** This distinction matters in real programs. If a value comes from **user input** (like a calculator app), you often declare the variable first and define it later — once the input is received.

14. Keywords

Definition: Keywords are **reserved words** in Java that already have a special meaning in the language's syntax. You cannot use them as identifiers (variable names).

Examples you already know: `class`, `public`, `static`, `void`

Java has **68 reserved keywords** in total.

★ Interesting Fact

A few keywords like `goto` and `const` are reserved in Java but were **never actually used** in the language. They were kept reserved because they're used in C/C++, and Java's designers wanted to avoid future naming conflicts.

⚠ Common Mistake

Trying to name a variable `class`, `int`, or any other keyword causes a **compilation error**, since Java gets confused between your identifier and the reserved syntax word.

15. Comments

Comments are ignored by the compiler — they exist purely to help humans understand the code.

```
// This is a single-line comment

/*
    This is a
    multi-line comment
*/
```

✅ **Best Practice:** Use comments to explain *why* something is done, not just *what* is being done — the code itself already shows the "what."

★ One-Page Revision

Core Definitions

- **Variable** = a named container in memory holding a value
- **Identifier** = the name of the variable
- **Literal** = the actual value stored
- **Static typing** = Java requires you to declare a variable's data type upfront
- **Declaration** = announcing a variable's type + name
- **Definition** = assigning it a value
- **Keyword** = a reserved word with special meaning (68 total in Java)

Primitive Data Types Cheat Sheet

Type	Size	Range / Values
<code>byte</code>	8-bit	−128 to 127
<code>short</code>	16-bit	−32,768 to 32,767
<code>int</code>	32-bit	−2 ³¹ to 2 ³¹ −1
<code>long</code>	64-bit	−2 ⁶³ to 2 ⁶³ −1
<code>float</code>	32-bit	Single precision decimal
<code>double</code>	64-bit	Double precision decimal (default)
<code>char</code>	16-bit	Single Unicode character
<code>boolean</code>	—	<code>true</code> or <code>false</code> only

Key Rules to Remember

- n bits → 2ⁿ unique combinations → max unsigned value = 2ⁿ − 1
- Decimal literals default to `double` ; add `f` for `float`
- `char` values are stored internally as integers, then binary
- Java uses 16-bit **Unicode**, not 8-bit ASCII, for global language support
- `boolean` accepts **only** `true` / `false` — never `0` / `1`
- Number system prefixes: `0b` binary, `0` octal, `0x` hexadecimal
- Underscores in numeric literals are ignored by the compiler (but not next to `.` or `e`)



Practice Questions

Basic

1. What is the difference between an identifier and a literal?
2. Why is Java called a "statically typed" language?
3. What is the size (in bits) of each integer type: `byte` , `short` , `int` , `long` ?

Medium 4. Why can a `byte` variable not store the value 255, even though 255 fits in 8 bits? 5. What's the difference between declaring and defining a variable? Give an example of each. 6. Convert the decimal number 13 into its binary representation.

Interview 7. Why does Java use Unicode instead of ASCII for the `char` type? 8. Why do float literals require an `f` suffix, but double literals don't need anything? 9. Why does Java strictly disallow `boolean b = 1;` unlike some other languages?

Exam-Oriented 10. Write Java code to declare one variable of each primitive type and print all their values. 11. Explain, with an example, how the same value (5) can be written in decimal, binary, octal, and hexadecimal in Java. 12. What are Java keywords? Name two keywords that exist but are never actually used in code, and explain why they're reserved.



Frequently Asked Questions

Q: Can I use a keyword as a variable name if I really need to? No. Keywords are strictly reserved — using one as an identifier causes a compilation error.

Q: Why does Java prefer `double` over `float` ? Modern processors are highly optimized for `double` , so it's more accurate and often just as fast. `float` is mostly kept for legacy compatibility.

Q: Is `0` the same as `false` in Java, like in C++? No. Java's `boolean` type only accepts the literals `true` and `false` — numeric values are not interchangeable with booleans.

Q: Do I need to memorize all 68 keywords? No — you'll naturally recognize them as you write more Java code. Focus on understanding *why* they're reserved rather than memorizing the full list.